DISTRIBUTED SYSTEMS

Lab 5

Nuno Preguiça, Sérgio Duarte, Kevin Gallagher, Salvatore Signorello, João Bordalo, Dina Borrego

GOALS

In the end of this lab you should be able to:

- Generalise the logic of the Server.
- Generalise the logic of the Client.

GOALS

In the end of this lab you should be able to:

- **Generalise the logic of the Server.**
- Generalise the logic of the Client.

GENERALISING THE LOGIC OF THE SERVER

In the project (and in general) you might want to provide the same service through different (but equivalent) interfaces…

Let's consider the case of providing the functionality of the Users service both through REST and gRPC.

GENERALISING THE LOGIC OF THE SERVE

REST

```
public class UsersResource implements RestUsers {

@Override
public String postUser(User user) {
    Log.info("postUser : " + user);
    // Check if user data is valid
    if (user.getName() == null || user.getPwd() == null || user.getDisplayName() == null ||
            user.getDomain() == null) {
        Log.info("User object invalid.");
        throw new WebApplicationException(Status.BAD_REQUEST);
    }
    try {
        hibernate.persist(user);
    } catch (ConstraintViolationException e) {
        e.printStackTrace(); //This exception is due to the user already existing...
        Log.info("User already exists.");
        throw new WebApplicationException(Status.CONFLICT);
    } catch( Exception x) {
        x.printStackTrace(); // Un-expected exception.Signal internal server error.
        throw new WebApplicationException(Status.INTERNAL_SERVER_ERROR);
    }
    return user.getName() + "@" + user.getDomain();
}
}
```

GENERALISING THE LOGIC OF THE SERVE

REST

```
public class UsersResource implements RestUsers {

@Override
public String postUser(
    Log.info("postUser
    // Check if user da
    if (user.getName()
            user.getDom
        Log.info("User
        throw new WebAp
    }
    try {
        hibernate.persi
    } catch (Constraint
        e.printStackTra
        Log.info("User
        throw new WebAp
    } catch( Exception
        x.printStackTra
        throw new WebAp
    }
    return user.getName
}
}
```

GRPC

```
public class GrpcUsersServerController implements GrpcUsersGrpc.AsyncService, Binda
    ...
    @Override
    public void postUser(GrpcUser user, StreamObserver<PostUserResult> responseObserver) {
        Log.info("postUser : " + user.getName() + ...);
        // Check if user data is valid
        if (user.getName() == null || user.getPwd() == null || user.getDisplayName() == null
                || user.getDomain() == null) {
            responseObserver.onError(errorCodeToStatus(ErrorCode.BAD_REQUEST));
            return;
        }

        try {
            hibernate.persist(DataModelAdaptor.GrpcUser_to_User(user));
            responseObserver.onNext(PostUserResult.newBuilder()
                        .setUserAddress(user.getName() + "@" + user.getDomain()).build());
            responseObserver.onCompleted();
        } catch (Exception e) {
            e.printStackTrace(); //Most likely due to the user already existing...
            responseObserver.onError(errorCodeToStatus(ErrorCode.CONFLICT));
        }
    }
}
```

GENERALISING THE LOGIC OF THE SERVE

REST

```
public class UsersResource implements RestUsers {

@Override
public String postUser(
    Log.info("postUser
    // Check if user do
    if (user.getName()
            user.getDom
        Log.info("User
        throw new WebAp
    }
    try {
        hibernate.persi
    } catch (Constrain
        e.printSt
        Log.info(
        throw new
    } catch( Excep
        x.printSt
        throw new
    }
    return user.ge
}
}
```

GRPC

```
public class GrpcUsersServerController implements GrpcUsersGrpc.AsyncService, Binda
    ...
    @Override
    public void postUser(GrpcUser user, StreamObserver<PostUserResult> responseObserver) {
        Log.info("postUser : " + user.getName() + ...);
        // Check if user data is valid
        if (user.getName() == null || user.getPwd() == null || user.getDisplayName() == null
                || user.getDomain() == null) {
            responseObserver.onError(errorCodeToStatus(ErrorCode.BAD_REQUEST));
                                                                        ).build());
                                                                        ...
        }
    }
}
```

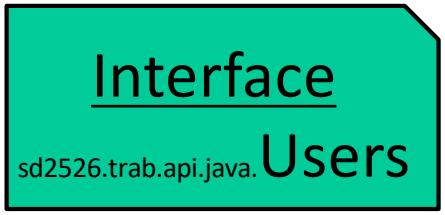
The logic for this service is extremely similar in both implementations. We should avoid repeating the code logic of this service, which would be harder to maintain, and to avoid accidentally adding errors across different implementations.

The challenge is that there are a few important differences:

- Parameters do not have the same types;
- Returning a result is different.

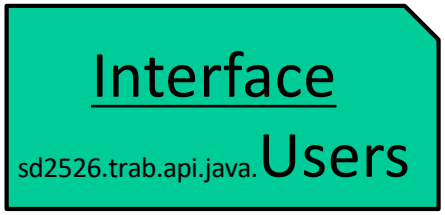
Generalising the logic of the Server

For the project, we suggest you adopt the following approach to generalise the logic of the server.



Generalising the logic of the Server

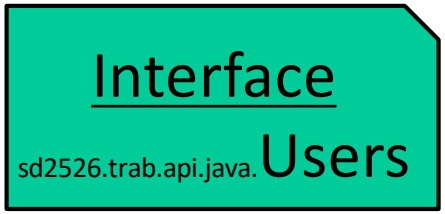
For the project, we suggest you adopt the following approach to generalise the logic of the server.




```
public interface Users {
    ...
    Result<String> postUser(User user);
    Result<User> getUser(String userId, String password);
    Result<User> updateUser(String userId, String password, User user);
    Result<User> deleteUser(String userId, String password);
    Result<List<User>> searchUsers(String pattern);
}
```

Generalising the logic of the Server

For the project, we suggest you adopt the following approach to generalise the logic of the server.

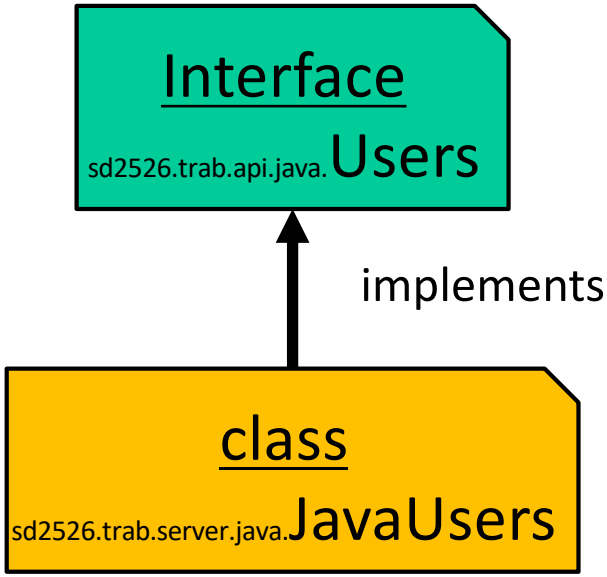



```
public interface Users {
    ...
    Result<String> postUser(User user);
    Result<User> getUser(String userId, String password);
    Result<User> updateUser(String userId, String password, User user);
    Result<User> deleteUser(String userId, String password);
    Result<List<User>> searchUsers(String pattern);
}
```

Simple java methods (no REST annotations) that always return the Result class (parameterized with the type returned by the method in case of success). This allows to generalize error handling between REST implementations and gRPC implementations.

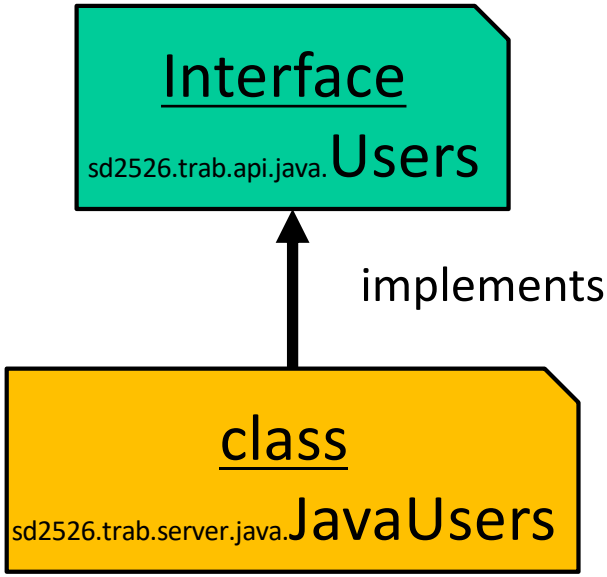
GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.



Generalising the logic of the Server

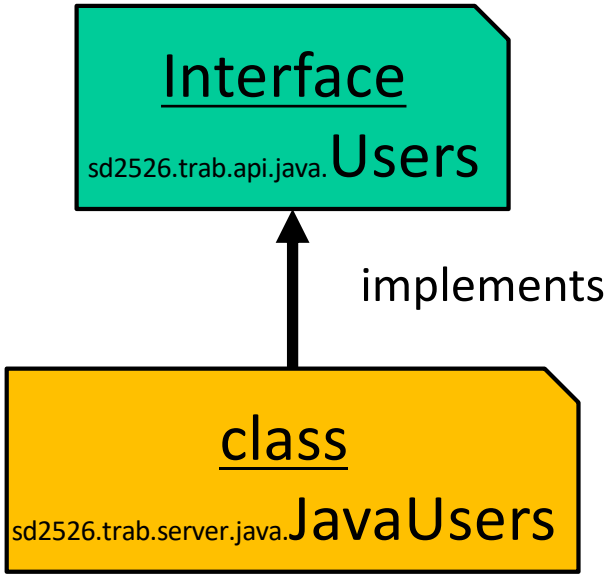
For the project, we suggest you adopt the following approach to generalise the logic of the server.




```java
public class JavaUsers implements Users {

    private Hibernate hibernate;

    public JavaUsers() {
        hibernate = Hibernate.getInstance();
    }

    @Override
    public Result<String> postUser(User user) {

       if (user.getName() == null … ) {
            Log.info("User object invalid.");
            return Result.error(ErrorCode.BAD_REQUEST);
       }
       try {
            hibernate.persist(user);
       } catch (ConstraintViolationException e) {
            return Result.error(ErrorCode.CONFLICT);
       }

       return Result.ok(user.getName()+"@"+user.getDomain());
    }
    ...
}
```

Generalising the logic of the Server

For the project, we suggest you adopt the following approach to generalise the logic of the server.

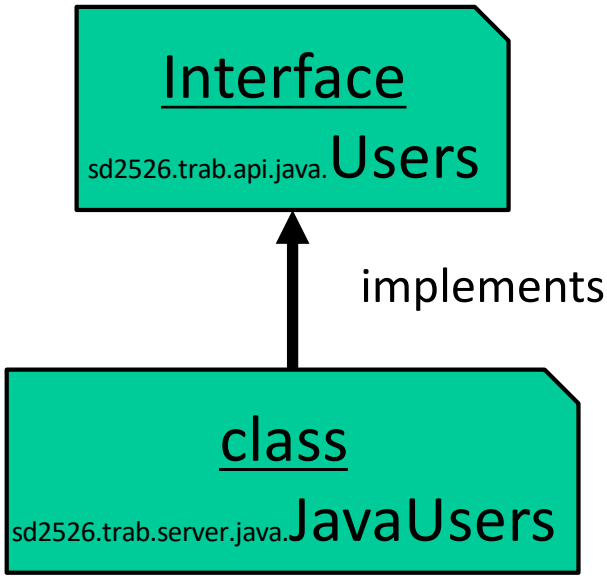


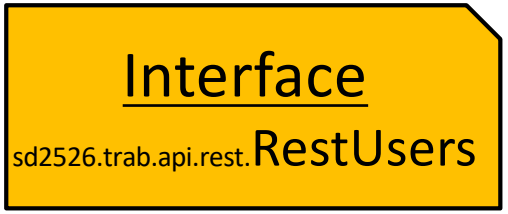
Provides an implementation that is agnostic to the way we expose the service.

```
public class JavaUsers implements Users {

    private Hibernate hibernate;

    public JavaUsers() {
        hibernate = Hibernate.getInstance();
    }

    @Override
    public Result<String> postUser(User user) {

       if (user.getName() == null … ) {
            Log.info("User object invalid.");
            return Result.error(ErrorCode.BAD_REQUEST);
       }
       try {
            hibernate.persist(user);
       } catch (ConstraintViolationException e) {
            return Result.error(ErrorCode.CONFLICT);
       }

       return Result.ok(user.getName()+"@"+user.getDomain());
    }
    ...
}
```

GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.

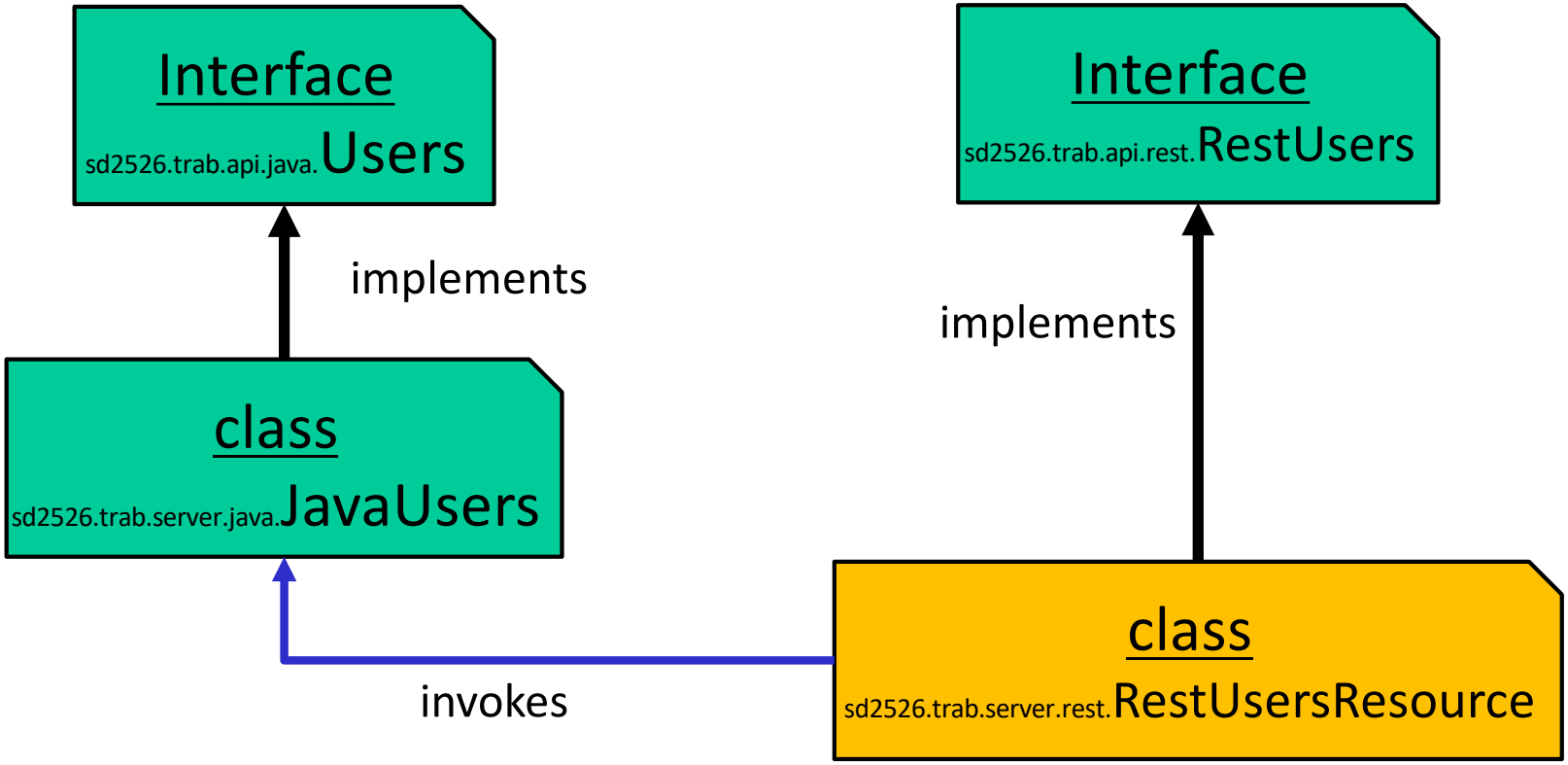




Specifies the REST service (does not return Result) and contains JAX-RS annotations.

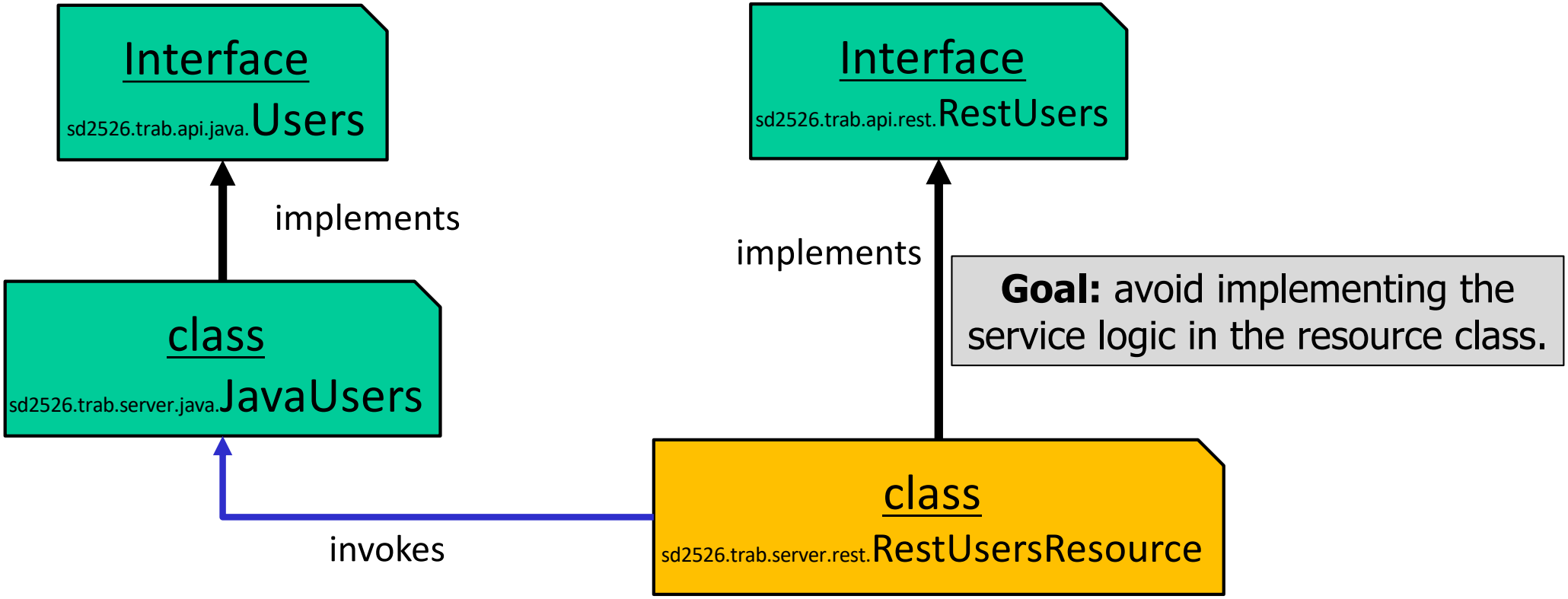
GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.



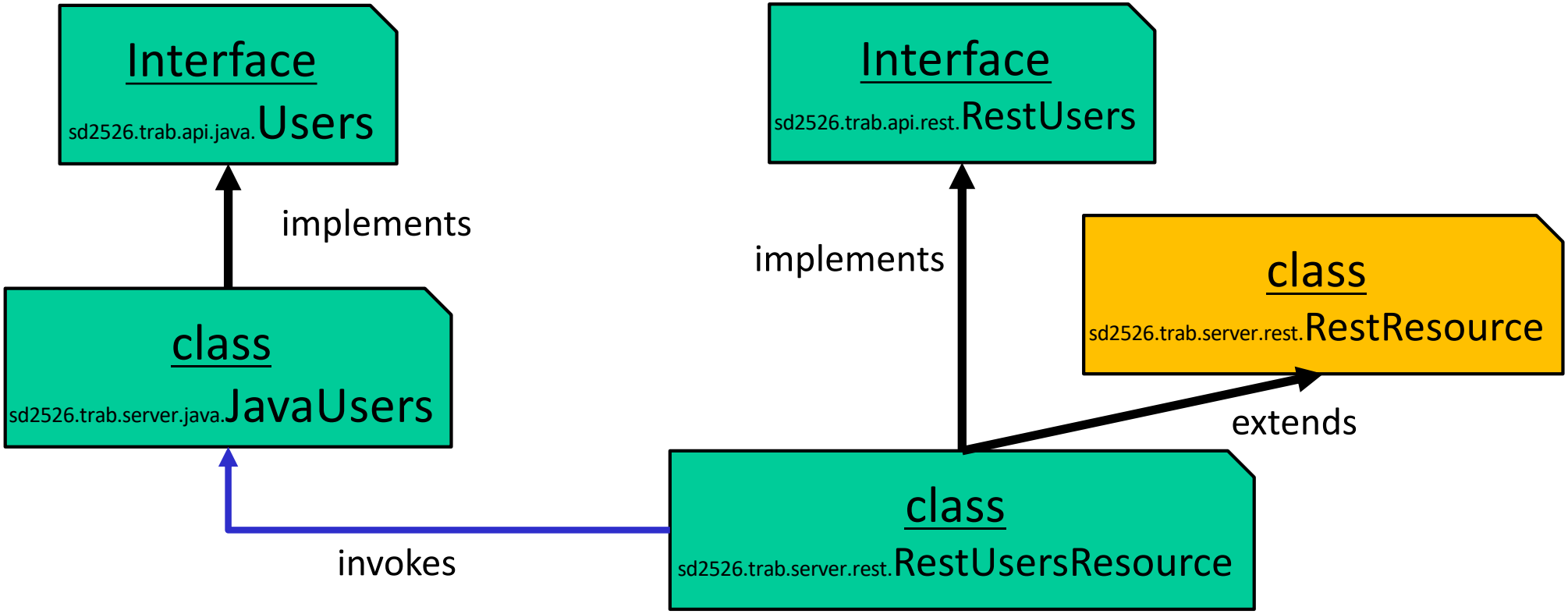
GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.



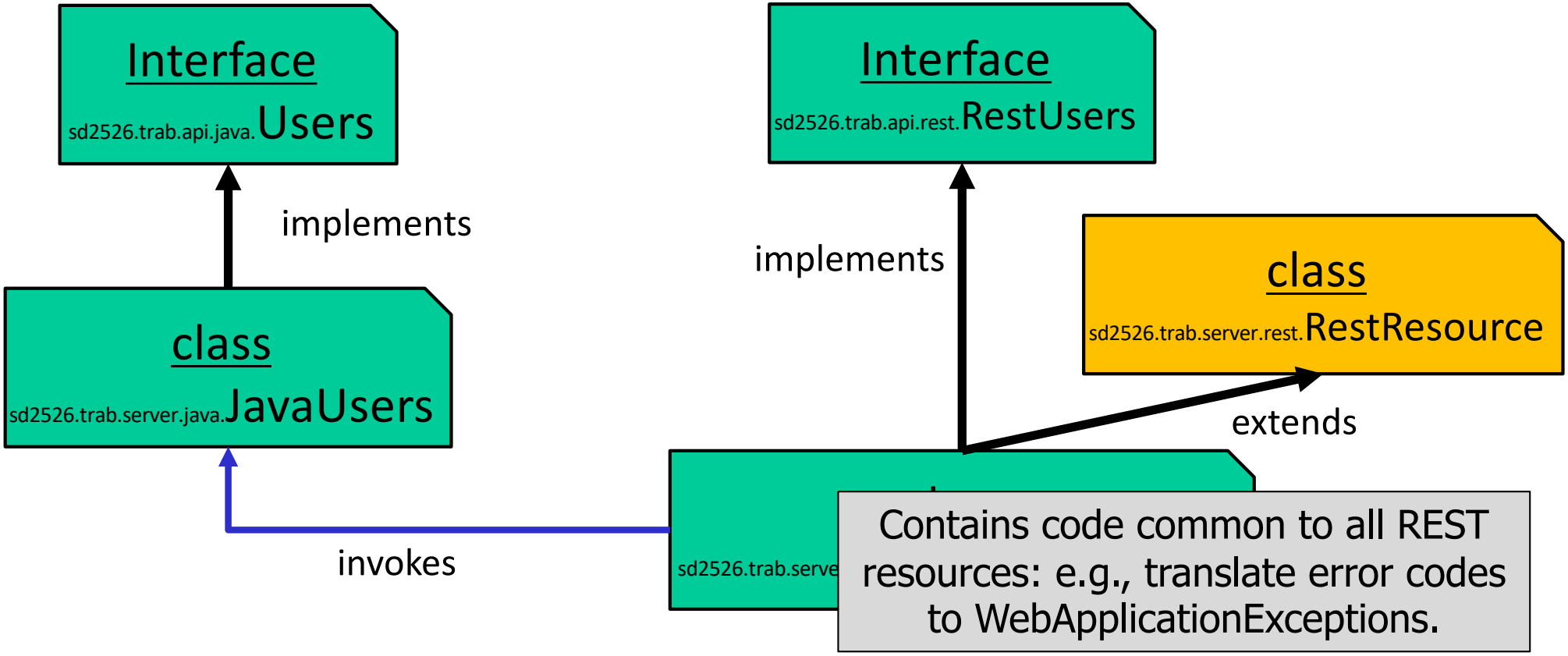
GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.



GENERALISING THE LOGIC OF THE SERVER

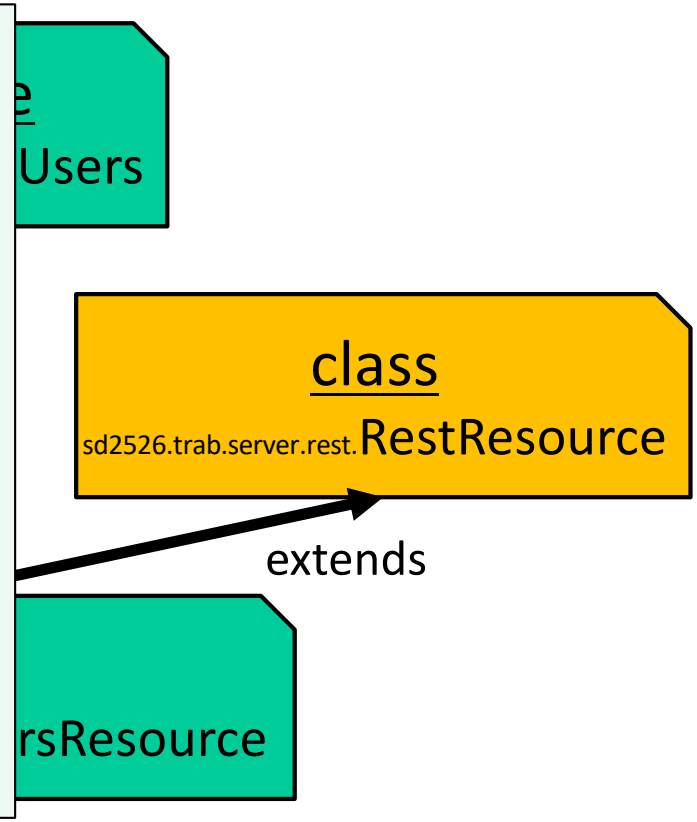
For the project, we suggest you adopt the following approach to generalise the logic of the server.



GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.

```
public class RestResource {

  private static Status errorCodeToStatus( Result.ErrorCode error ){
     Status status =  switch( error) {
       case NOT_FOUND -> Status.NOT_FOUND;
       case CONFLICT -> Status.CONFLICT;
       case FORBIDDEN -> Status.FORBIDDEN;
       case NOT_IMPLEMENTED -> Status.NOT_IMPLEMENTED;
       case BAD_REQUEST -> Status.BAD_REQUEST;
       default -> Status.INTERNAL_SERVER_ERROR;    };
     return status;
  }

  protected static <T> T unwrapResultOrThrow( Result<T> result ) {
    if( result.isOK() )
      return result.value();
    else {
      var status = errorCodeToStatus( result.error() );
      throw new WebApplicationException( status );
    }
}
```



GENERALISING THE LOGIC OF THE SERVER

```
public class RestUsersResource extends RestResource implements RestUsers {
  …
  final Users impl;

  public RestUsersResource() {
     impl = new JavaUsers();
  }

  @Override
  public String postUser(User user) {
     Log.info("postUser : " + user);
     return super.unwrapResultOrThrow( impl.postUser( user ) );
  }

  @Override
  public User getUser(String name, String pwd) {
     Log.info("getUser : user = " + name + "; pwd = " + pwd);
     return super.unwrapResultOrThrow( impl.getUser(name, pwd));
  }
  …
}
```

sd2526.trab.server.java.JavaUsers

extends

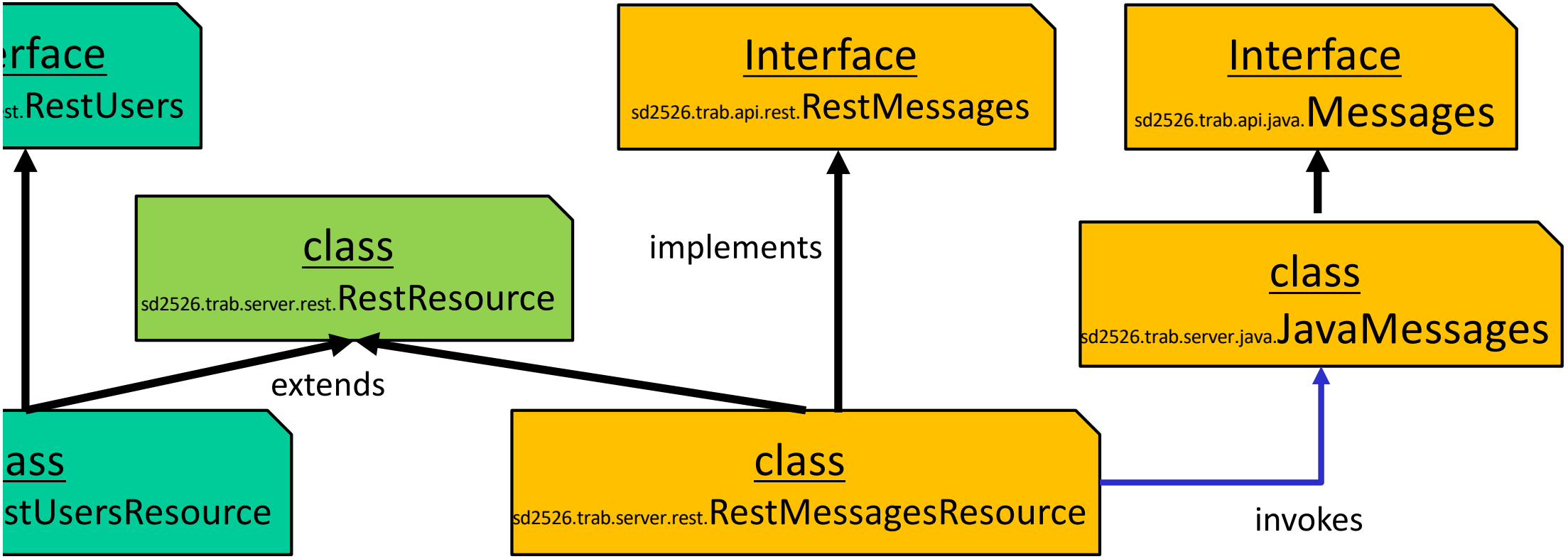
invokes

class

sd2526.trab.server.rest.RestUsersResource

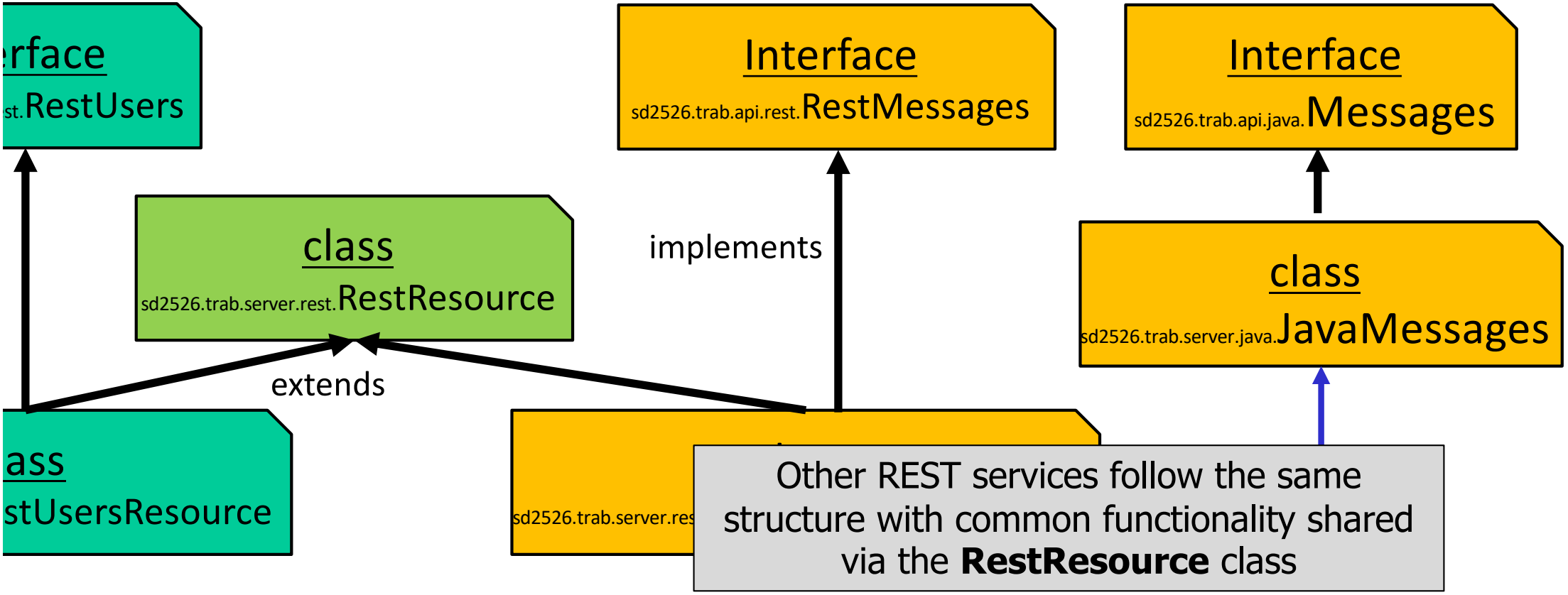
GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.



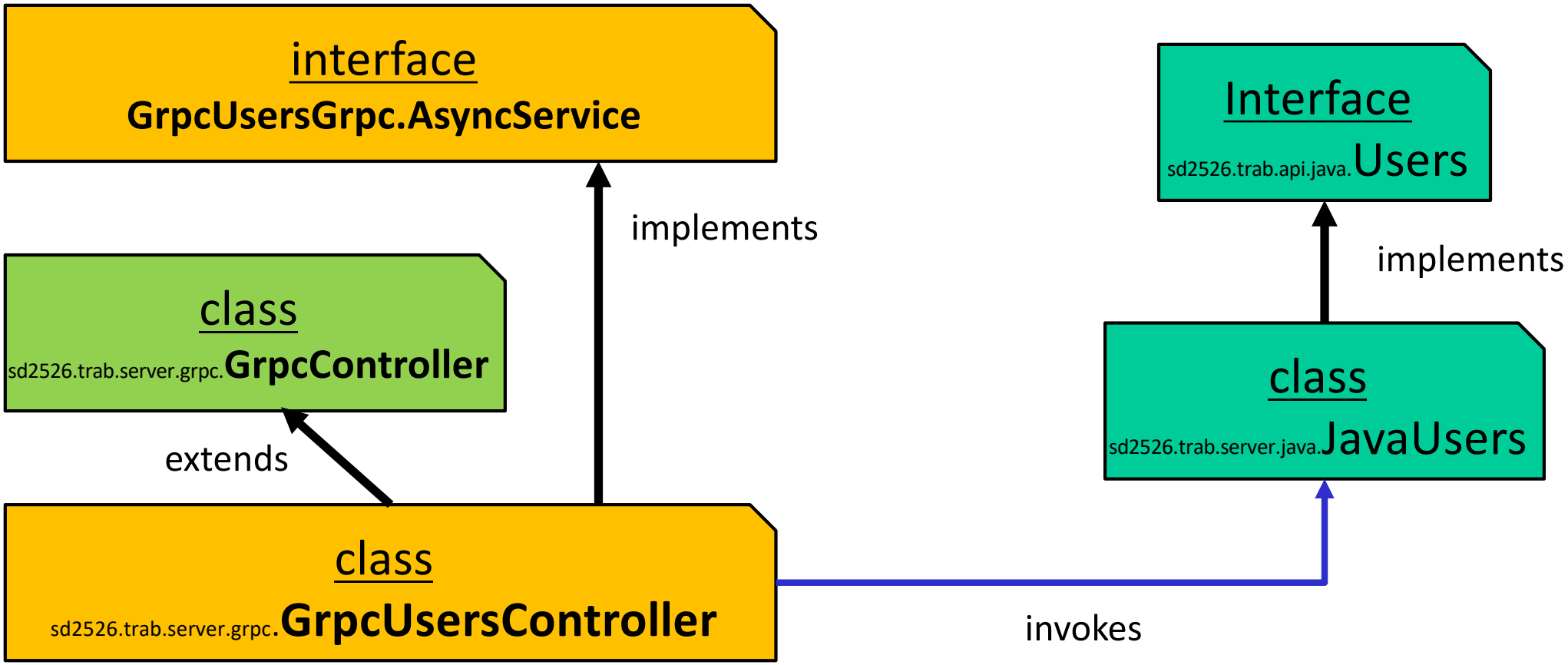
GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.



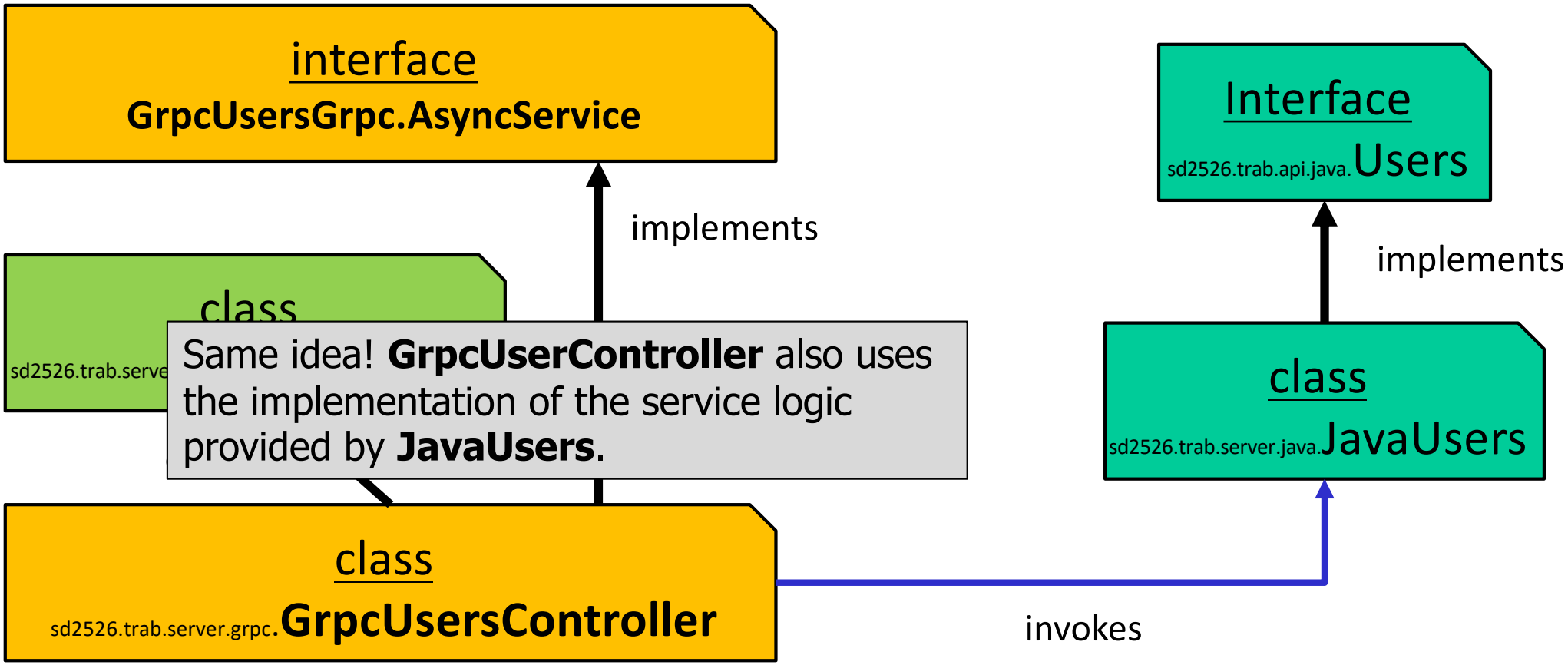
Generalising the logic of the Server

For the project, we suggest you adopt the following approach to generalise the logic of the server.



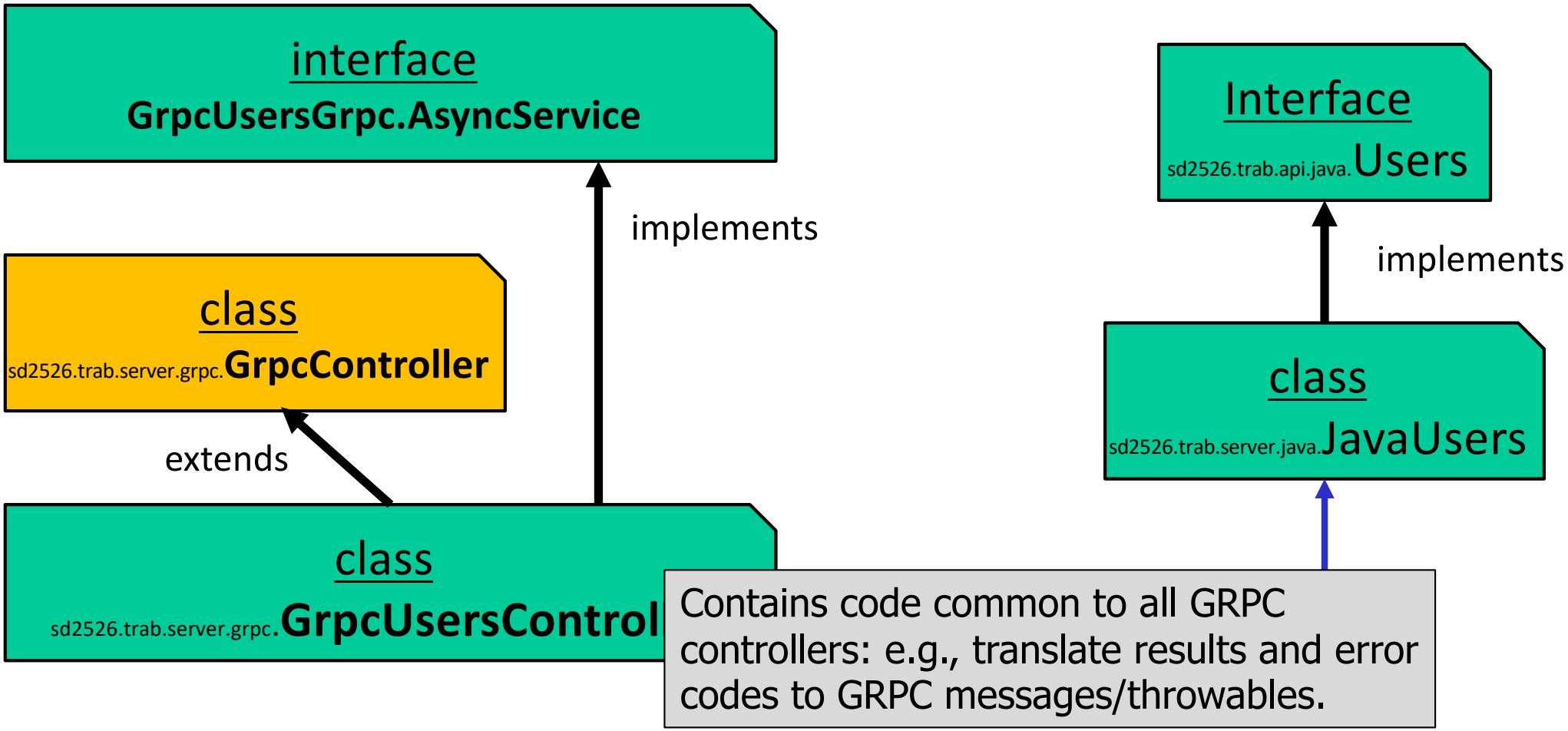
GENERALISING THE LOGIC OF THE SERVER

For the project, we suggest you adopt the following approach to generalise the logic of the server.



Generalising the logic of the Server

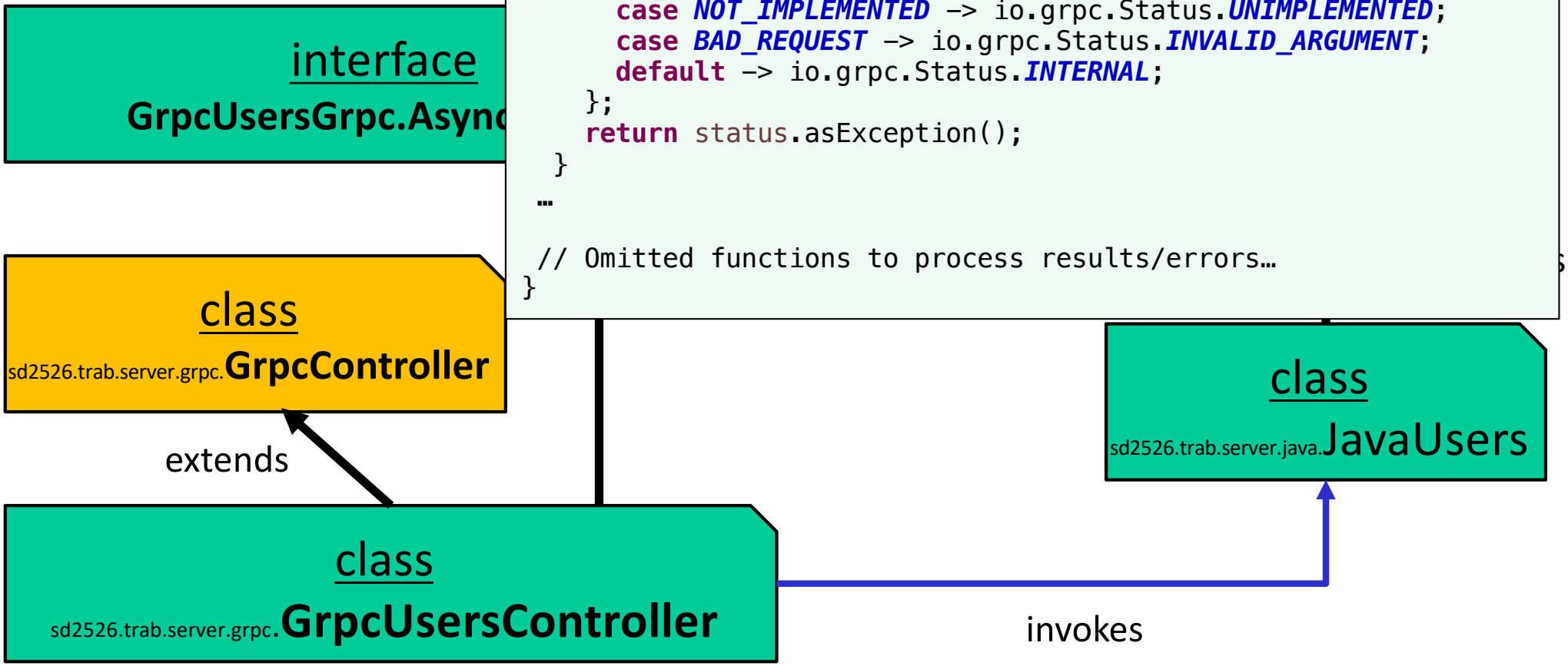
For the project, we suggest you adopt the following approach to generalise the logic of the server.



GENERALISING THE LOGIC OF THE SERVER

For the project, we su
to generalise the logi

```
public class GrpcController {

 protected static Throwable errorCodeToStatus(ErrorCode error) {
    var status =  switch( error) {
      case NOT_FOUND -> io.grpc.Status.NOT_FOUND;
      case CONFLICT -> io.grpc.Status.ALREADY_EXISTS;
      case FORBIDDEN -> io.grpc.Status.PERMISSION_DENIED;
      case NOT_IMPLEMENTED -> io.grpc.Status.UNIMPLEMENTED;
      case BAD_REQUEST -> io.grpc.Status.INVALID_ARGUMENT;
      default -> io.grpc.Status.INTERNAL;
    };
    return status.asException();
  }
 …

 // Omitted functions to process results/errors…
}
```



GENERALISING THE LOGIC OF THE SERVER

```
public class GrpcUsersController extends GrpcController
                                    implements GrpcUsersGrpc.AsyncService, BindableService {
   Users impl = new JavaUsers();

   @Override
   public final ServerServiceDefinition bindService() {
       return GrpcUsersGrpc.bindService(this);
   }

   @Override
   public void postUser(GrpcUser user, StreamObserver<PostUserResult> responseObserver) {
       super.toGrpcResult(responseObserver, impl.postUser(GrpcUser_to_User(user)),
             (userAddress) -> PostUserResult.newBuilder().setUserAddress(userAddress).build());
   }

   @Override
   public void getUser(GetUserArgs request, StreamObserver<GetUserResult> responseObserver) {
       super.toGrpcResult(responseObserver, impl.getUser(request.getName(), request.getPwd()),
              (user) -> GetUserResult.newBuilder().setUser(User_to_GrpcUser(user)).build());
   }
  …
```

ents

sd2526.trab.server.java.JavaUsers

extends

class

sd2526.trab.server.grpc.**GrpcUsersController**

invokes

GOALS

In the end of this lab you should be able to:

- Generalising the logic of the Server.
- **Generalising the logic of the Client.**
  - **Generalising the type of the Client.**
  - Avoiding duplication of code on retries.

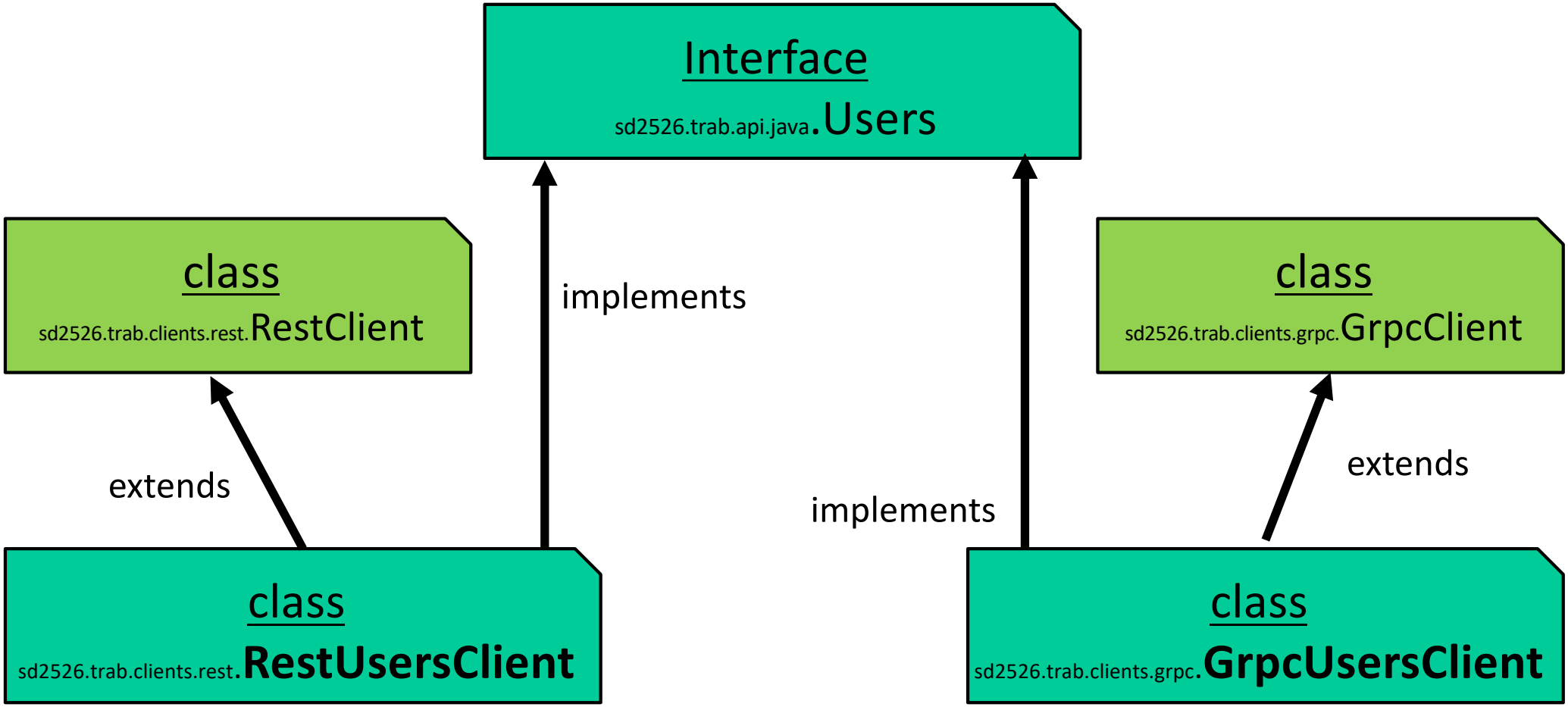
Generalising The Type of the Client

In the project, it is supposed to be possible to have a configuration that mixes REST and GRPC servers.

To allow interacting with each type of server, it is a good idea to generalise the type of client.

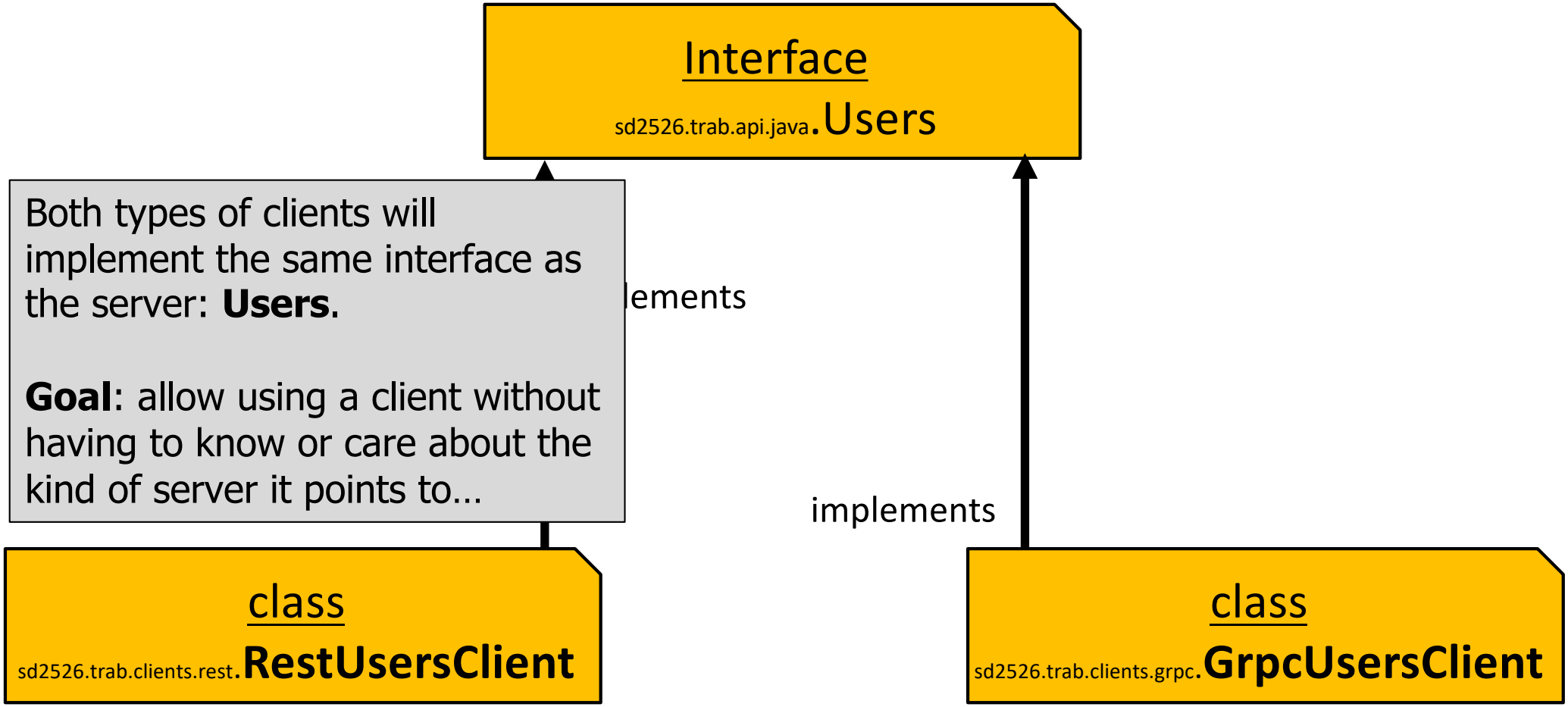
Generalising The Type of the Client

The same principles can be applied to the client-side of the service.



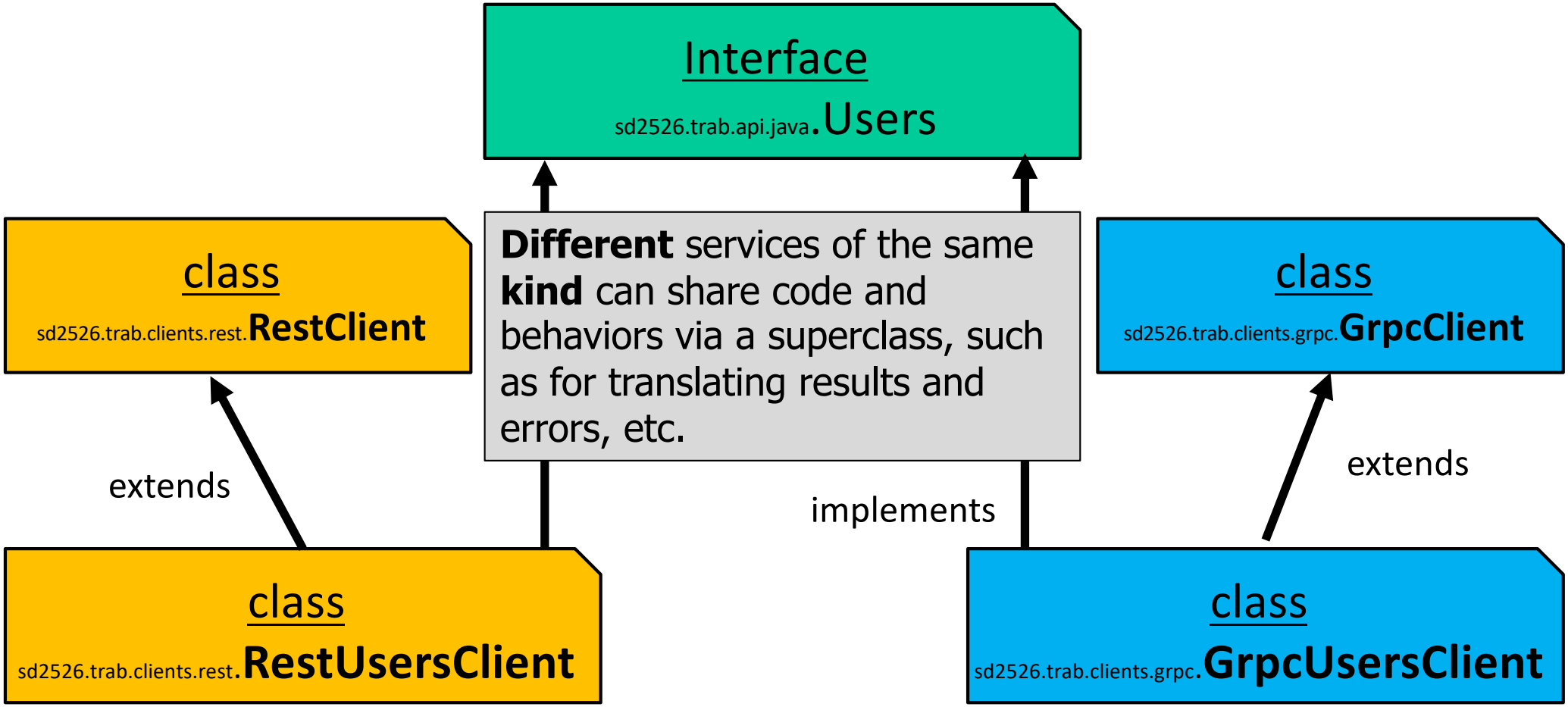
Generalising The Type of the Client

The same principles can be applied to the client-side of the service.



Generalising The Type of the Client

The same principles can be applied to the client-side of the service.



Generalising The Type of the Client

The same principles can be applied to the client-side of the service.

class

sd2526.trab.clients.rest.**RestClient**

```
public class RestClient {

   private static final int READ_TIMEOUT = 5000;
   private static final int CONNECT_TIMEOUT = 5000;

   final Logger logger;

   final URI serverURI;
   final Client client;
   final ClientConfig config;

   protected WebTarget target;

   public RestClient(URI serverURI, Logger logger) {
     this.serverURI = serverURI;
     this.logger = logger;

     this.config = new ClientConfig();
     this.config.property(ClientProperties.READ_TIMEOUT, READ_TIMEOUT);
     this.config.property(ClientProperties.CONNECT_TIMEOUT, CONNECT_TIMEOUT);

     this.client = ClientBuilder.newClient(config);
     this.target = client.target(serverURI);
   }
```

Rest clients for different services can share most Webtarget setup logic, including timeout values…

Generalising The Type of the Client

The same principles can be applied to the client-side of the service.

```
public class RestClient {
  …

   private static final int MAX_RETRIES = 3;
   private static final long RETRY_SLEEP = 1000;


   protected <T> Result<T> reTry(Supplier<Result<T>> func) {

      for (int i = 0; i < MAX_RETRIES; i++) {
         try {
            return func.get();
         } catch (ProcessingException x) {
           Thread.sleep(RETRY_SLEEP);
         } catch (Exception x) {
            return Result.error(ErrorCode.INTERNAL_ERROR);
         }
      }
      return Result.error(ErrorCode.TIMEOUT);
   }
 …
```

class
sd2526.trab.clients.rest.**RestClient**

The code for handling transient communication failures can be also factored out. The trick is to pass a function that does the actual request and returns the result: **see** java.util.function.Supplier

Generalising The Type of the Client

The same principles can be applied to the client-side of the service.

class

sd2526.trab.clients.rest.**RestClient**

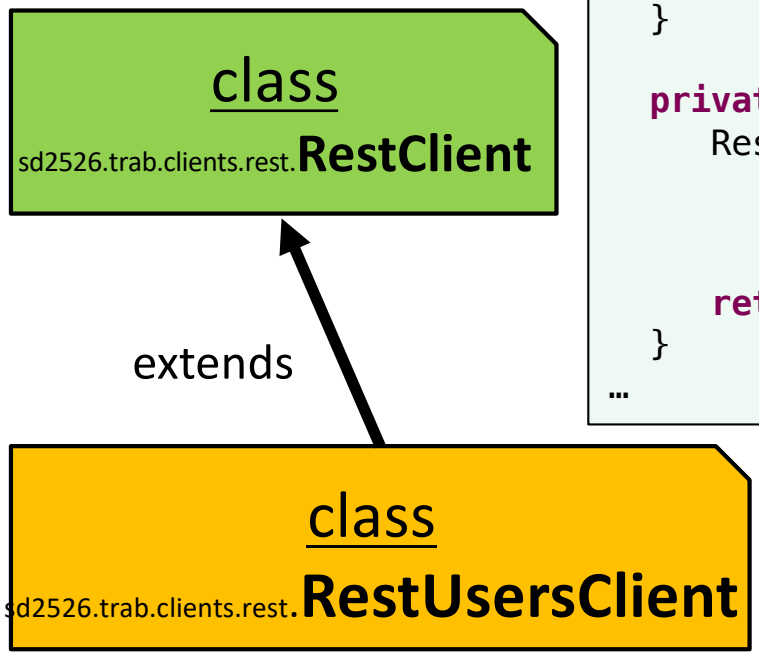
```
public class RestClient {

…

protected <T> Result<T> processResponse(Response r, Class<T> entityType) {
    try {
       var status = r.getStatusInfo().toEnum();
       if (status == Status.OK && r.hasEntity())
          return Result.ok(r.readEntity(entityType));
       else
          if( status == Status.NO_CONTENT)
              return Result.ok();

       return Result.error(getErrorCodeFrom(status.getStatusCode()));
   } finally {
      r.close();
   }
}

…
}
```

Code for processing REST responses to Result instances can also be shared among clients…

Generalising The Type of the Client

The same principles can be applied to the client-side of the service.

```
public class RestUsersClient extends RestClient implements Users {

  public RestUsersClient( URI serverURI ) {
     super( serverURI, Log );
     target = super.target.path( RestUsers.PATH );
  }

  public Result<String> postUser(User user) {
     return super.reTry( () -> doPostUser( user ) );
  }

  private Result<String> doPostUser(User user) {
     Response r = target.request()
                  .accept( MediaType.APPLICATION_JSON)
                  .post(Entity.entity(user, MediaType.APPLICATION_JSON));

     return super.processResponse(r, String.class);
  }
…
```



GOALS

In the end of this lab you should be able to:

- Generalise the logic of the Server.
- **Generalise the logic of the Client.**
  - Generalise the type of the Client.
  - **Using a client factory to abstract the server type from the caller…**

Client Factory : Specific to each Service

Factory for creating clients for the Users service…

```
public class UsersClientFactory {

   private static final String REST = "/rest";
   private static final String GRPC = "/grpc";
   private static final Object DOMAIN_DELIMITER = "@";

   static public Users get(String domain) {
       var sn = "%s%s%s".formatted(Users.SERVICE_NAME, DOMAIN_DELIMITER, domain);
       return newClient(Discovery.knownUrisOf(sn, 1)[0] );
   }

   static private Users newClient( URI serverURI ) {
       var path = serverURI.getPath();
       if (path.endsWith(REST))
           return new RestUsersClient(serverURI);

       if (path.endsWith(GRPC))
           return new GrpcUsersClient(serverURI);

       throw new Runtim
   }
}
```

The URI of each service obtained via Discovery can be used to decide which client is needed. By convention, if the path ends with **/rest,** we want a REST client; ending with **/grpc**, means the server is GRPC and we need to create a GRPC client.

Client Factory : Specific to each Service

How to use the factory…

```
Users client = UsersClientFactory.get("some_domain");

Result<User> u = client.getUser("name", "pwd");

if( u.isOK() )
     System.out.println("Got user: " + u.value() );
else
     System.out.println("Got error:" + u.error() );
```

Using this approach, this factory can return clients for a specific service, abstracting away the kind of server that is being called.

We need to implement a factory for each service we have.

CLIENT FACTORY: GENERIC ALTERNATIVE

It is possible to have a fully generic factory, which returns clients for any service.

```
public class ClientFactory<T> {

  final String serviceName;
  final Function<URI, T> restClientFunc;
  final Function<URI, T> grpcClientFunc;

  ClientFactory( String serviceName, Function<URI, T> restClientFunc, Function<URI, T> grpcClientFunc) {
     this.restClientFunc = restClientFunc;
     this.grpcClientFunc = grpcClientFunc;
     this.serviceName = serviceName;
  }

  private T newClient( URI serviceURI ) {
    var path = serviceURI.getPath();
    if (path.endsWith(REST))
       return restClientFunc.apply( serviceURI );
    if (path.endsWith(GRPC))
    return grpcClientFunc.apply( serviceURI );
    throw new Runtime
  }
}
```

A generic factory needs to know: which **service name** to use with Discovery to find the server URI; and, needs **two functions** to know how to create the clients for each service type (REST or GRPC).

CLIENT FACTORY: GENERIC ALTERNATIVE

How to use the factory…

```
var UsersClients = new ClientFactory<Users>( Users.SERVICE_NAME,
                                             RestUsersClient::new,
                                             GrpcUsersClient::new );
Users users = UsersClients.get("some_domain")
…
```

```
var UsersClients = new ClientFactory<Users>( Users.SERVICE_NAME,
                                             (uri) -> new RestUsersClient( uri ),
                                             (uri) -> new GrpcUsersClient( uri );
Users users = UsersClients.get("some_domain")
…
```

Both versions of the code above are equivalent.

The factory uses ***Users.SERVICE_NAME*** to discover the service URI.

The factory also receives the two functions, passed as lambdas, needed create the actual clients, depending on their type.

These functions are simply the constructors of the client classes: **RestUsersClient**, ***GrpcUsersClient***, which take the server URI as the sole argument.